Transformers: https://hackage.haskell.org/package/transformers

Resources
ReaderT design pattern: https://www.fpcomplete.com/blog/readert-design-pattern/
Exception best practices (ExceptT): https://www.fpcomplete.com/blog/exceptions-best-practices-haskell/

Transformer
Control.Monad.Trans.Class:https://hackage.haskell.org/package/transformers/docs/Control-Monad-Trans-Class.html

```
-- Control.Monad.Trans.Class
class (forall m. Monad m => Monad (t m)) => MonadTrans t
where
  lift :: Monad m => m a -> t m a
```

Monad transformer
A monad transformer is a **wrapper** to create a monad from another monad
i.e. **composition**
e.g. MaybeT
  if m * is a monad then MaybeT m = m (Maybe *) is a monad

```
-- Basic
--
-- lift: lift inner monad in a nested monad.
-- i.e. `return` analogue for inner monad
return :: Monad (t m) => a -> t m a
  -- lift inner value a -> t m a
lift :: (Monad m, MonadTrans t) => m a -> t m a
  -- lift inner monad m a -> t m a
return 1 :: MaybeT [] Int               -- [Just 1]
return [1] :: MaybeT [] Int             -- NO. [Just
[1]]
lift [1,2] :: MaybeT [] Int             -- [Just 1,Just
2]
return 1 :: MaybeT ((->) Int) Int       -- f(x)=Just 1
return (+1) :: MaybeT ((->) Int) Int    -- NO. f(x)=Just
(+1)
lift (+1) :: MaybeT ((->) Int) Int      -- f(x)=Just
(x+1)
```

==========
======
MaybeT

```
newtype MaybeT m a = MaybeT { runMaybeT :: m (Maybe a) }

instance Monad m => MonadTrans(MaybeT m)
instance Monad m => Monad(MaybeT m)
```

```
instance Monad m => MonadPlus(MaybeT m)
instance Monad m => Alternative(MaybeT m)
instance MonadFix m => MonadFix(MaybeT m)
instance MonadFail m => MoandFail(MaybeT m)
```

MaybeT
Monad with exit i.e. computation stops if Nothing
e.g.
MaybeT [] a like [Maybe a]
MaybeT ((->) a) b like (a -> Maybe b)

```
---- Basic
--
MaybeT :: Monad m => m (Maybe a) -> MaybeT m a     --
MaybeT constructor
mzero :: MaybeT m a                                --
Always short circuit
mzero = MaybeT $ pure Nothing

f :: MaybeT ((->) Int) Int
f = MaybeT (\x -> Just (2*x))       -- i.e. (*2) wrapped
in Maybe
f = MaybeT (Just.(*2))
f = MaybeT (*2)                     -- ERROR
f = MaybeT (\x -> Nothing)          -- Always short
circuit
f = MaybeT $ pure Nothing           -- same
f = mzero                           -- same
f = MaybeT (\x -> let               -- Conditional
  y = x*2
  in if (y<10) then Just y else Nothing)
f = do                              -- with do
expression
  y <- lift (*2)
  if (y<20) then return y else mzero
```

```
-- Basic example
f,f0,g,g' :: MaybeT ((->) Int) Int
f = MaybeT Just.(*2)
f0 = mzero
g = do
  x <- lift (*2)
  let y = x*2
  in if (y<10) then return y else Nothing
g' = (+1) <$> g

runMaybeT f 1         -- Just 2
runMaybeT f 5         -- Just 10
runMaybeT f0 1        -- Nothing
runMaybeT f0 5        -- Nothing
runMaybeT g 1         -- Just 2
runMaybeT g 4         -- Just 8
runMaybeT g 5         -- Nothing
```

```
runMaybeT g' 1        -- Just 3
runMaybeT g' 1        -- Just 9
runMaybeT g' 5        -- Nothing
```

```
-- MonadTrans (MaybeT m)
--
lift :: Monad m => m a -> MaybeT m a
f = lift (*2)       -- MaybeT (Just.(*2))
f = lift [1,2]      -- MaybeT [Just 1,Just 2]


-- e.g. list
l :: MaybeT [] Int
l = MaybeT [Just 1]
l = MaybeT [Nothing]
l = lift [1..5]           -- MaybeT [Just 1..Just 5]
l = mzero                 -- MaybeT [Nothing]
l = do                    -- MaybeT [Just 1, Just 2,
Nothing, Nothing, Nothing]
 x <- lift [1..5]
 unless (x < 3) mzero
 return x


-- Monad m => Functor (MaybeT m)
-- Monad m => Applicative (MaybeT m)
--
-- e.g. (-> r)
f1,f2 :: MaybeT ((->) Int) Int
f1 = lift (+1)
f2 = lift (*2)
(*3) <$> f1                   -- 3(x+1) in MaybeT
pure (*) <*> f1 <*> f2        -- (x+1)*(2x) in MaybeT

-- e.g. list
l1,l2 :: MaybeT [] Int
l1 = lift [1,2,3]
l2 = lift [4,5]
(*2) <$> l1                   -- MaybeT [Just 2,Just 4,
Just 6]
pure (*) <*> l1 <*> l2        -- MaybeT [Just 1*4,Just
1*5,Just 2*4, ... Just 3*5]
```

==========

Except (Either)

Resources
Exception best practices (ExceptT): https://www.fpcomplete.com/blog/exceptions-best-practices-haskell/

```
newtype ExceptT e m a = ExceptT (m (Either e a))

instance MonadTrans (ExceptT e)
instance Functor m => Functor (ExceptT e m)
instance (Functor m, Monad m) => Applicative (ExceptT e
m)
instance Monad m => Monad (ExceptT e m)
instance MonadFix m => MonadFix (ExceptT e m)
instance MonadFail m => MonadFail (ExceptT e m)
instance (Monad m, Monoid e) => MonadPlus (ExceptT e m)


runExceptT :: ExceptT e m a -> m (Either e a)
mapExceptT :: (m (Either e a) -> n (Either e' b)) ->
ExceptT e m a -> ExceptT e' m b
withExceptT :: Functor m => (e -> e') -> ExceptT e m a
-> ExceptT e' m a


handleE :: Monad m =>  (e -> ExceptT e' m a) -> ExceptT
e m a -> ExceptT e' m a          -- transformer >= 0.6
tryE :: Monad m => ExceptT e m a -> ExceptT e m a
(Either e a)                           -- transformer >=
0.6
finallyE :: Monad m => ExceptT e m a -> ExceptT e m ()
-> ExceptT e m a                  -- transformer >= 0.6
```

Except (Either)
Monad with error.

```
---- Basic
--
throwE :: e -> ExceptT e m a
throwE "OOPS"
runExcept (throwE "OOPS")            -- Left "OOPS"


-- catchE : analogous to catch block in try-catch
-- i.e. can either suppress or rethrow error
--
catchE :: Monad m => ExceptT e m a -> (e -> ExceptT e' m
a) -> Except e' m a
handleE :: Monad m => (e -> ExceptT e' m a) -> ExceptT e
m a -> ExceptT e' m a
c :: ExceptT e' m a
catchE c (\e -> ... )
```

```
// Analogous in JS
try { c } catch (e) { ... }
```

```
-- e.g. suppress error
catchE c (\e -> return 1)
```

```
// Analogous in JS
try { c } catch (e) { return 1 }
```

```
-- e.g. rethrow error
catchE c (\e -> throwE e)
catchE c (\e -> throwE "FATAL")
```

```
// Analogous in JS
try { c } catch (e) { throw e }
```

```
-- e.g.
c0 = return 1 :: Except e Int
c1 = throwE "OOPS"
catchE c0 (\e -> return 100)         -- Except (Right 1)
i.e. unchanged
catchE c1 (\e -> return 100)         -- Except (Right
100)
catchE c0 (\e -> throwE "FATAL")     -- Except (Right 1)
i.e. unchanged
catchE c1 (\e -> throwE "FATAL")     -- Except
(Left "FATAL")


-- Execution
runExcept :: Except e m a -> Either e a
runExceptT :: ExceptT e m a -> m (Either e a)
runExcept (return 1)             -- (Right 1)
runExcept (throwE "Oops")        -- (Left "Oops")

mapExcept :: ((Either e a) -> (Either e' b)) -> Except e
a -> Except e' b
mapExceptT :: (m (Either e a) -> n (Either e' b)) ->
ExceptT e m a -> ExceptT e' m b
withExcept :: (e -> e') -> Except e a -> Except e' a
withExceptT :: Functor m => (e -> e') -> ExceptT e m a
-> ExceptT e' m a


-- Utils
--
tryE :: Monad m => ExceptT e m a -> ExceptT e m (Either
e a)                          -- transformer >=0.6
finallyE :: Monad m => (ExceptT e m a -> ExceptT e m ())
-> ExceptT e m a            -- transformer >=0.6
```

```
-- same as tryE
tryE m = catchE (Right <$> m) (return . Left)

-- same as finallyE
finallyE m closer = do
  v <- catchE m (\e -> closer >> throwE e)
  closer >> return v
```